

HTML dinámico

En esta actividad instalaremos y exploraremos cómo generar HTML dinámico desde el back-end.

 Individual



Práctica: EJS

Consulta la guía práctica completa en el Docusaurus del curso

 [Ir a la práctica](#)



Objetivos

- ✓ Generar desde el back-end, interfaces dinámicas HTML5 para el cliente por medio de un motor de `templates` y que incorporen `partials`.



Instrucciones

1 Recuerda que para generar una aplicación es necesario inicializar el proyecto con `npm init`, e instalar `express` y `body-parser` con `npm install --save [nombre_paquete]`. Si es necesario, recuerda configurar el archivo `package.json` para incluir el script que quieres que se ejecute con `npm start`.

2 Sigue la demostración del profesor en la sesión de clase.

3 Para devolver como respuesta un archivo html desde express, se requiere el módulo `path`, puedes incorporarlo con `const path = require('path');` al inicio de tus archivos de código.

Por convención, los archivos HTML se guardan en un directorio llamado `views`, en referencia a la capa de la vista de un estilo arquitectónico MVC. Para devolver un archivo HTML, se realiza de la siguiente forma:

```
router.get('/una-ruta', (request, response, next) => {  
  response.sendFile(path.join(__dirname, '..', 'views', 'el-  
  archivo.html'));  
});
```

La función `join` sirve para armar la ruta, y la ventaja que tiene sobre hacerlo manualmente, es que considera el sistema operativo donde el código resida para que no tengas problemas si estás en MacOS, Linux o Windows.

La variable global `__dirname`, contiene el directorio del sistema de archivos donde se encuentra tu aplicación, y el resto de los argumentos son cada uno de los directorios para llegar al archivo. Observa que después de `__dirname`, el argumento es `'..'` debido a que si estás en una carpeta de alguno de tus módulos, subirás un nivel en los directorios para llegar al nivel raíz y de ahí viajar a la carpeta `views`, para finalmente llegar al archivo `el-archivo.html`.

4 Por defecto, nuestro servidor no puede entregar un CSS como respuesta, entonces es necesario decirle que cuando un documento HTML requiere de un CSS, este archivo se mande de manera estática. Para ello debemos de configurar una carpeta de archivos estáticos, por convención la llamaremos `public`, y al folder donde pondremos los archivos css lo llamaremos `css`. Para poder incluirlos dentro de los HTML lo podemos hacer con el método `static` de `express`:

```
app.use(express.static(path.join(__dirname, 'public')));
```

5 Utilizando alguno de tus laboratorios anteriores (o si lo prefieres empieza de 0), elabora una aplicación web con las siguientes características:



- La aplicación debe poder responder al menos a 5 rutas diferentes, distribuidas en al menos 2 módulos, y mandar una respuesta HTTP 404 cuando la ruta no existe.
- En alguna de las rutas, la aplicación debe contener al menos 1 forma que debe enviar datos al servidor por POST. El servidor debe reaccionar ante este envío y guardar los datos en un archivo de texto dentro del mismo servidor.

6 Vamos a utilizar **EJS** como motor de *templates* para generar html de manera dinámica. Este motor lo estaremos usando en los ejemplos, pero eres libre de utilizar alguno diferente en tus laboratorios y proyecto.

7 Para instalar EJS: `npm install --save ejs`.

8 Para configurar EJS en Express, debemos indicarle a Express que vamos a utilizar como motor para la capa de las vistas EJS, y en seguida indicar por medio de una variable de configuración de Express, la carpeta donde estarán almacenados los archivos html correspondientes a las vistas, por convención, utilizaremos la carpeta `views`:

```
app.set('view engine', 'ejs');
app.set('views', 'views');
```

9 Los archivos EJS (`.ejs`), son archivos que en su mayoría contienen código HTML, pero que también permiten escribir código JS, utilizando tags con el símbolo `%` de la siguiente forma: `<% código de javascript %>`.

Para desplegar un `template` de EJS, lo hacemos con el método `render`, y como argumento ponemos el nombre del archivo `.ejs` sin poner la extensión del archivo:

```
router.get('/una-ruta', (request, response, next) => {
  response.render('vista_ejs');
});
```

Para escribir una variable en la vista, primero debemos de pasarle a la vista un objeto de javascript con los nombres y valores de las variables:

```
router.get('/una-ruta', (request, response, next) => {
  response.render('vista_ejs', {variable: valor});
});
```

Y finalmente, para imprimir la variable en la vista utilizamos el operador `=`:

```
<%= nombre_variable %>
```

10 Así como imprimimos variables, también podemos poner código de estructuras de control, por ejemplo:

```
<% if (arreglo.length > 0) { %>
  <ul>
    <% for (let elemento of arreglo) { %>
      <li><%= elemento.atributo %></li>
    <% } %>
  </ul>
<% } else { %>
  <h1>El arreglo está vacío</h1>
<% } %>
```

11 Refactoriza tu aplicación para que mantenga la funcionalidad pero ahora trabaje con un motor de vistas.

12 Mejora tu aplicación para que en lugar de guardar los datos en un archivo, ahora los despliegue en una de las páginas.

13 Como has podido experimentar, mantener las 5 páginas implica estar copiando y pegando pedazos de código HTML, lo cual es un proceso propenso a errores. Los motores de vistas como EJS, permiten reutilizar código sin la necesidad de estar copiando y pegando. Una estrategia para ello, es la utilización de `partials`, que son pedazos reutilizables de código de una vista. Por convención, dentro de la carpeta `views`, crearemos una carpeta `includes` en donde pondremos nuestros `partials`. Para incluir un partial en una vista:

```
<%- include('includes/head.ejs') %>
```

Observa el operador `-`. A diferencia del operador `=` que te protege de una inyección de código, el operador `-` no lo hace, pero esto es lo que permite que se inserte el código HTML de nuestro `partial`.

14 Refactoriza tu aplicación para hacer un uso efectivo de los partials con aspectos como el código del `head` de tu aplicación, y quizás de la barra de navegación (si cuentas con una) y del footer.



Preguntas

1 ¿Qué otros templating engines existen para node?



Recursos

 [pug](#)

 [Handlebars](#)

 [EJS](#)

 [Cross-site Scripting \(XSS\)](#)



Entrega

A través de tu repositorio personal.

Enlaces importantes

[Solicitud de asesoría \(Denisse / Alex\)](#)

[Políticas de trabajo en equipo](#)

[Guía de integridad académica del MIT](#)

[Malla de evaluación](#)

[Agenda de entrevistas](#)